TokenCake: A KV-Cache-centric Serving Framework for LLM-based Multi-Agent Applications

Zhuohang Bian
Peking University
Beijing, China
22373017@buaa.edu.cn

Feiyang Wu
Peking University
Beijing, China
2501111907@stu.pku.edu.cn

Zhuoran Li
Peking University
Beijing, China
2200012710@stu.pku.edu.cn

Teng Ma
Alibaba Group
Beijing, China
sima.mt@alibaba-inc.com

Youwei Zhuo
Peking University
Beijing, China
youwei@pku.edu.cn

arXiv:2510.18586v4 [cs.DC] 21 Aug 2026

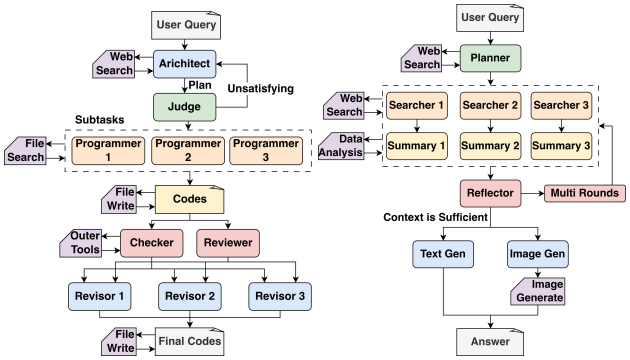
Abstract

Large Language Models (LLMs) are increasingly deployed in complex multi-agent applications that rely on external function calls. This workload creates severe performance challenges for the KV Cache: spatial contention leads to the eviction of critical agents' caches and temporal underutilization leaves the cache of agents stalled on long-running function calls idling in GPU memory.

We present TokenCake, a KV-Cache-centric serving framework that bridges this gap by co-optimizing scheduling and memory management through an agent-aware design. TokenCake's Temporal Scheduler employs an event-driven, opportunistic policy to proactively offload idle KV Caches during function calls and uses predictive uploading to hide data transfer latency. TokenCake's Spatial Scheduler uses dynamic memory partitioning, guided by a hybrid priority metric combining graph structure and runtime state, to reserve GPU memory for critical-path agents. Our evaluation on representative multi-agent benchmarks shows that TokenCake reduces end-to-end latency by over 47.06% and improves effective GPU memory utilization by up to 16.9% compared to vLLM.

1 Introduction

Large Language Models (LLMs) are powerful reasoning engines, and applications built upon them are evolving from single-response generation to complex, multi-agent systems. This evolution has enabled powerful applications in domains like autonomous code generation [17], complex financial analysis [26], and realistic environment simulation [20]. The defining characteristic of these applications is a dual-interaction model: frequent external **agent-tool** interaction through function calls, and complex internal **agent-agent** collaboration through structured workflows. Externally, agents use function calls to interact with tools, data sources, and APIs. Internally, multiple specialized agents collaborate within a dependency graph to solve a larger problem.



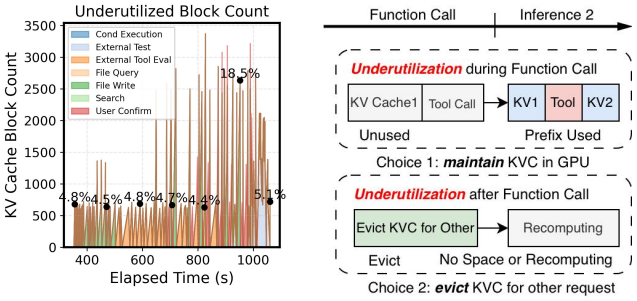
(a) Multi-agent Coding **(b)** Deep Research

Figure 1. Example LLM-based Multi-Agent Applications. Each colored box represents a specialized agent. Purple boxes denote function calls to external tools.

Figure 1 illustrates this application model with two representative examples. Code-Writer [3, 5, 8, 11, 18, 24, 28] orchestrates a pipeline of agents, including programmers, reviewers, and testers, that make frequent external calls to tools like file systems and code interpreters. Deep-Research [6, 19, 23] uses a workflow of agents that search, summarize, and synthesize information, requiring external calls to web search APIs and document stores. The combination of complex internal dependencies and frequent, long-running external interactions produces workload patterns that are fundamentally different from traditional LLM inference. These patterns introduce critical performance challenges for the underlying serving infrastructure that existing systems are not designed to handle.

These challenges are rooted in inefficient KV Cache management, which manifests along two dimensions: **temporal underutilization** and **spatial contention**.

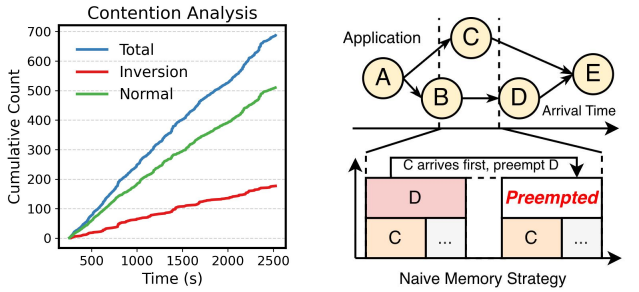
Temporal underutilization roots in the frequent and long-running function calls inherent to agentic workloads. An agent's execution follows an *LLM Inference1* $\Rightarrow$ *Function Call* $\Rightarrow$ *LLM Inference2* pattern, during which its KV Cache—the exact prefix needed for the subsequent inference—sits idle



(a) Idle KV Cache blocks due to external function call.

(b) The lifecycle of an agent's KV Cache during a function call.

Figure 2. The Temporal Underutilization Problem.



(a) Preemption events over time of the Code Writer workload.

(b) KV Cache blocks held by non-critical agents.

Figure 3. The Spatial Contention Problem.

in GPU memory. Figure 2b illustrates this lifecycle: after the first inference phase, the agent's KV Cache remains resident while it waits for the external tool to respond. This forces a difficult trade-off: retain the cache and waste GPU resources that could serve active requests, or evict it and incur a costly recomputation when the agent resumes. This inefficiency is substantial. As shown in Figure 2a, at peak moments, as much as 18.5% of the GPU KV Cache pool is occupied by stalled agents, directly reducing the system's capacity for active computation. Systems like Teola [22] have identified the latency challenge posed by this pattern and propose a workflow-level optimization. Teola intelligently pipelines the execution of LLM and non-LLM micro stages, aiming to overlap the tool-use-time of one agent with the computational work of another. While this approach effectively hides latency by improving the application's end-to-end execution schedule, it is fundamentally compute-centric. Teola's scheduler optimizes the flow of operations but remains blind to the state of the underlying GPU memory resources. Consequently, the KV Cache of the stalled agent continues to idly occupy valuable GPU memory, a problem that is exacerbated when multiple agents stall concurrently.

Spatial contention arises as numerous agents compete for limited GPU memory. As illustrated in Figure 3b, an agent on the critical path can be stalled if a non-critical agent arrives earlier and occupies the limited GPU memory. Agent-unaware allocation policies like FCFS lead to a problem we term *critical inversion*: a non-critical agent occupies memory and causes the eviction of a critical-path agent's KV Cache. This forces the evicted critical agent to undergo a costly context recomputation, stalling the entire application workflow. We measure this phenomenon on the Code-Writer application and count the critical inversion events over time. As shown in Figure 3a, these harmful preemptions occur frequently under realistic workloads. While systems like Parrot [14] and Autellix [16] are workflow-aware, their optimizations operate at the request level, agnostic to the fine-grained KV Cache contention between individual agents. They optimize the order and batching of requests but do not manage the underlying memory allocation. Consequently, even with an optimal schedule, a high-throughput, non-critical task group identified by Parrot could still occupy GPU memory and inadvertently cause the eviction of a latency-sensitive, critical agent's KV Cache. This exposes a fundamental limitation: scheduling optimization alone cannot solve memory contention.

To address these challenges, we present TokenCake, a KV-Cache-centric serving framework that co-optimizes scheduling and memory management through an agent-aware design. TokenCake begins with a frontend API that allows users to define an application's internal agent collaborations and external tool interactions as a graph (§3.1). This graph enables two specialized schedulers to manage the KV Cache lifecycle with application-level context. To mitigate temporal underutilization, the **Temporal Scheduler** uses an event-driven, opportunistic policy to proactively offload the KV Cache of stalled agents during function calls and uses predictive uploading to hide transfer latency (§4). To resolve spatial contention, the **Spatial Scheduler** employs a dynamic memory partitioning policy, guided by a hybrid priority metric, to reserve memory for critical-path agents (§5).

The two schedulers share a unified pressure-aware coordination protocol that captures GPU capacity, reserved capacity, waiting demand, offloadable stalled blocks, and pending upload debt. This shared protocol prevents the two schedulers from optimizing against different notions of pressure and ensures that every memory movement is justified by a concrete scheduling benefit.

Our evaluation on representative multi-agent benchmarks (§7) shows that TokenCake reduces end-to-end latency by over 47.06% compared to vLLM under high load and improves GPU memory utilization by over 16.9%. Ablation studies confirm that agent-aware scheduling and temporal offload must work together: agent scheduling alone reduces critical-path latency under memory pressure, while selective offloading

Table 1. Latency characteristics of common tools in MCP.

Tool	Device	Latency	Variability
File System	CPU	100 ms	50ms
Git	CPU	100 ms	100ms–1s
Database (SQLite)	CPU	100–1000 ms	500ms
Web Search	CPU	1–5s	1–10s
AI Generation	GPU	5–30s	10–60s

helps when function call stalls expose a useful scheduling window. Neither mechanism subsumes the other, and applying offload without agent context can degrade performance.

2 Background And Motivation

2.1 LLM-Based Multi-Agent Applications

Agents. A common design pattern in agentic systems is the decomposition of complex problems into a series of sub-tasks handled by specialized agents [10, 12, 25]. These collaborations are often modeled as a Directed Acyclic Graph (DAG), where nodes represent agents and edges represent explicit dependencies. In these workflows, the dependencies are critically important; for instance, a reviewer agent cannot begin its task until a programmer agent has completed its code generation. Furthermore, not all agents contribute equally to the application's end-to-end latency. Some agents lie on the critical path, meaning any delay they experience directly increases the total time-to-result for the entire application.

Function Calls. A key feature of modern agents is the use of function calls to interact with external data sources or execute actions. These calls connect the LLM to a vast array of tools, such as database clients, code interpreters, or third-party web APIs. To standardize these interactions, the Model Context Protocol (MCP) [7] is emerging as an open standard, providing a unified interface and a rich ecosystem of pre-defined tools. As shown in Table 1, compiled from the official MCP documentation and our empirical measurements, the latencies of these common tools have a wide and often unpredictable distribution.

2.2 Limitations Of Existing Serving Systems

The challenges of spatial contention and temporal underutilization expose the limitations of two distinct categories of state-of-the-art serving systems: those that are agent-aware but compute-centric, and those that are KV-Cache-centric but agent-agnostic.

A class of recent work has made serving systems agent-aware by incorporating the application's DAG into the scheduling logic. Systems like Parrot[14] and Autellix[16] use this graph to mitigate head-of-line blocking by prioritizing critical requests, while Teola[22] optimizes the execution pipeline for an individual agent's interaction with external tools.

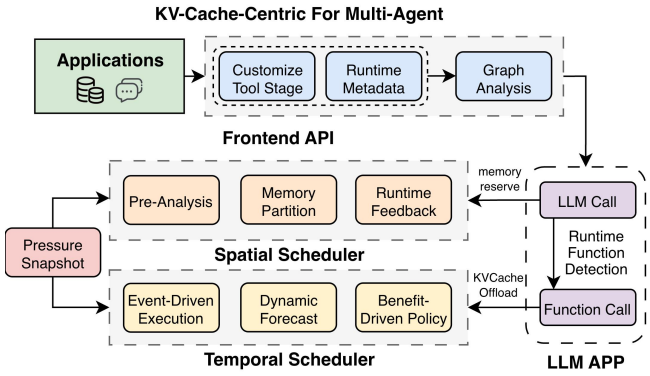


Figure 4. TokenCake Overview.

While these approaches improve high-level orchestration, their focus is fundamentally compute-centric. They optimize the order and batching of requests but do not manage the underlying memory allocation. Consequently, they cannot prevent critical inversion, as a high-throughput but non-critical task can still occupy GPU memory and cause the eviction of a critical agent's KV Cache. Furthermore, because they are not memory-centric, they do not address temporal underutilization, lacking the mechanisms to manage or repurpose the idle KV Cache of stalled agents.

Another line of work has focused on making serving more KV-Cache-centric, introducing advanced memory management and offloading policies. For instance, vLLM[13]'s PagedAttention solves internal memory fragmentation, while systems like Mooncake[21] and CachedAttention[9] have implemented offloading for general workloads.

However, while these systems are memory-aware, their policies are fundamentally agent-agnostic. They treat all KV Cache with equal importance, lacking the context to differentiate a critical-path agent from a non-critical one, which leaves them vulnerable to critical inversion. Furthermore, their offloading policies are typically reactive—triggered by memory pressure or session inactivity—rather than proactive. They are not designed to leverage the predictable idle periods during function calls to mitigate underutilization.

3 Overview

TokenCake is designed to optimize multi-agent application performance by managing KV Cache resources across both temporal and spatial dimensions. Figure 4 shows the three primary components of TokenCake: a Frontend API, a Temporal Scheduler, and a Spatial Scheduler.

The Frontend API (§3.1) translates a user's multi-agent application into a graph annotated with function call stages and time estimates. The Temporal Scheduler (§4) consumes this graph to manage the lifecycle of individual KV Cache blocks over time, offloading idle caches during function calls and uploading them before the agent resumes. The Spatial

```
1 rag_graph = Graph()
2 plan_node = LLMNode()
3 rag_node = RagNode(
4     stages=[Stage("query"), Stage("embed"),
  Stage("generate")],
5     predict_time=1, config=rag_config)
6 revise_node = ReviseNode()
7 output_node = LLMNode()
8 rag_graph.add_edge(plan_node, rag_node)
9 rag_graph.add_edge(rag_node, revise_node)
10 rag_graph.add_edge(revise_node, output_node)
```

Figure 5. Defining a multi-agent RAG application with the TokenCake API.

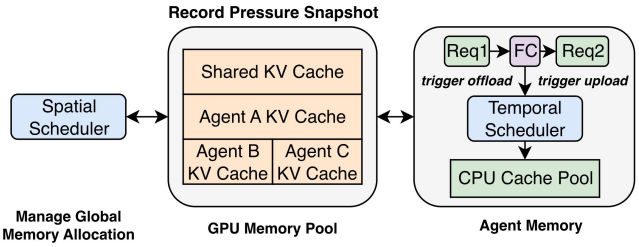


Figure 6. Coordination between the Spatial Scheduler and the Temporal Scheduler within a single scheduling step.

Scheduler (§5) partitions GPU memory dynamically based on both the static graph structure and runtime signals, reserving capacity for critical-path agents so that non-critical work cannot evict their KV Caches. Both schedulers are coordinated through a shared memory pressure-aware coordination protocol that gives them a consistent view of GPU capacity and prevents contradictory memory decisions.

3.1 Frontend API

TokenCake provides a programming interface that lets users describe a multi-agent application as a Directed Acyclic Graph (DAG). Nodes represent agents or computational units, and edges represent data dependencies. The API exposes three kinds of information that existing serving systems lack: graph structure, fine-grained function call stages, and performance metadata.

Figure 5 shows how a user builds a simple Retrieval Augmented Generation application. The `FuncNode` abstraction (line 4) lets users decompose a function call into multiple sequential stages. This decomposition gives the Temporal Scheduler a real-time view of function progress rather than a single start-to-finish interval, enabling more precise upload timing. The API also allows users to supply critical performance metadata directly within the graph definition. In line 5, the `predict_time` parameter provides an estimated execution time for the function call, which helps the Temporal Scheduler make more accurate decisions about when to offload and prefetch an agent's KV Cache.

3.2 Coordination between the Spatial and Temporal Schedulers

The Temporal Scheduler and the Spatial Scheduler optimize different dimensions of KV Cache management but compete for the same GPU memory. Without coordination, the Temporal Scheduler may upload a KV Cache into blocks reserved for a critical request, or the Spatial Scheduler may admit a request into blocks needed for an imminent upload. TokenCake prevents such conflicts through a shared pressure snapshot and a fixed execution order within each scheduling step, as illustrated in Figure 6.

The pressure snapshot captures GPU and CPU block availability, per-agent-type reserved capacity, waiting demand, offloadable stalled blocks, and pending upload debt. Both schedulers read this snapshot so that every memory movement has a concrete scheduling benefit: an offload occurs only when freed blocks can admit useful work, and an upload occurs only when the resumed request will not displace a more important active one.

Each scheduling step proceeds in four phases: (1) refresh application metadata and build the pressure snapshot; (2) update the Spatial Scheduler's reservation plan if the adjustment window has expired; (3) the Temporal Scheduler reserves GPU blocks for imminent uploads, transfers ready CPU-resident KV Caches back to the GPU, and evaluates newly stalled requests for offload; (4) the Spatial Scheduler forms the next batch under agent-aware admission control, routing each waiting request to shared capacity, reserved capacity, or deferral.

4 The Temporal Scheduler

When an agent issues a function call, its KV Cache sits idle in GPU memory until the external tool responds. The Temporal Scheduler converts this idle period into a productive scheduling window by offloading the stalled agent's KV Cache to CPU memory and uploading it back before the agent resumes, avoiding the costly recomputation that a naive eviction would require.

Table 2 contrasts TokenCake's policy with three prior systems. CachedAttention [9] offloads on session inactivity; Mooncake [21] offloads under memory pressure or LRU eviction; InferCept [1] reactively swaps KV Caches to CPU upon external call interception. None use the function call event as an explicit scheduling trigger, none differentiate caches by graph criticality, and none gate offload on whether freed blocks can admit useful work. An always-offload alternative saturates PCIe bandwidth and adds unnecessary overhead for short function calls. TokenCake avoids this through its opportunistic policy (§4.2), which offloads only when freed memory will be productively used.

Table 2. Comparison of KV Cache offloading and prefetching policies.

Category	Criteria	TokenCake	Mooncake	CachedAttention	InferCept
General	FC Aware	**Yes**	No	No	Partial
	Data Granularity	Block	Block	Layer	Block
Offload	Strategy	**Proactive**	Reactive	Reactive	Reactive
	Trigger	FC Start	Cache Pool Pressure	Session Inactive	Interception Signal
	Decision Logic	Cost-Benefit	LRU	Session	Min-Waste Heuristic
Prefetch	Strategy	**Predictive**	Proactive	Reactive	Reactive
	Trigger	Predicted FC Completion	SLO-based Schedule	Session Resumption	Interception End
	Decision Logic	Static + Dynamic	Static	Static	FCFS

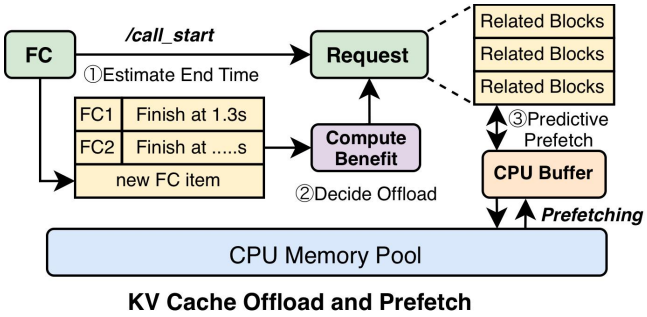


Figure 7. Lifecycle of the Temporal Scheduler's offload and predictive upload mechanism.

4.1 Event-Driven Offload And Predictive Upload

Figure 7 illustrates the lifecycle that the Temporal Scheduler manages for each stalled agent. The scheduler is driven by two runtime events and a dynamic forecasting model that connects them.

Runtime Event Loop. A *call_start* event fires when an agent emits a function call. The scheduler consults the opportunistic policy (§4.2) to decide whether offloading the agent's KV Cache is worthwhile. If the policy approves, the KV Cache is transferred to CPU memory asynchronously. As the predicted completion time approaches, the scheduler begins a gradual upload (§4.3) to hide transfer latency behind other computation. A *call_finish* event fires when the tool result returns. If the KV Cache has already been uploaded, the agent resumes immediately. If the tool returns earlier than predicted, the scheduler triggers an immediate upload to ensure correctness. The observed execution time is fed back to the forecasting model to refine future predictions.

Dynamic Forecasting. The scheduler maintains a per-function-type time estimate. When no history is available, the estimate defaults to the value supplied by the user through the graph definition, or to a conservative system-wide constant if no user estimate exists. After the first observed execution, the estimate transitions to an exponentially weighted moving average. When the user provides a time estimate (t_{user}) and the system has accumulated historical data ($t_{history}$), the model combines both:

$$t_{estimate} = \alpha \cdot t_{user} + (1 - \alpha) \cdot t_{history} \quad (1)$$

4.2 Opportunistic Policy For Proactive Offloading

Not every function call stall justifies an offload. Offloading is beneficial only when three conditions hold simultaneously: the predicted stall is long enough to cover a round-trip transfer, a waiting request exists that can use the freed GPU blocks, and the later upload can be prepared without displacing more important work. The opportunistic policy encodes these conditions as a gate that rejects offloads when the net benefit is unclear.

Transfer Overhead Estimation. The primary cost is the round-trip data transfer time between GPU and CPU memory. For a request holding N_{blocks} blocks, the transfer time is linear in N_{blocks} in TokenCake's scene:

$$T_{transfer} = T_{offload}(N_{blocks}) + T_{upload}(N_{blocks}) \quad (2)$$

Both terms are calibrated from measured per-block transfer constants on the target platform (§7.6).

Scheduling Window. The scheduler computes the available scheduling window as the predicted stall minus the transfer overhead: $T_{window} = T_{FC} - T_{transfer}$. If this window is non-positive, the offload is rejected immediately because the stall is too short.

Waiting-Request Fit. As shown in Algorithm 1, the scheduler converts the scheduling window into a token capacity estimate using the system's observed decode throughput. It then searches the waiting queue for a request whose incremental KV Cache demand fits within the freed blocks and whose total work fits within the token capacity. If no such request exists, offloading provides no immediate benefit and is rejected. The search uses a *first_fit* policy that selects the first eligible request in queue order. §7.5 compares *first_fit* with *best_fit* and *priority_first* alternatives and shows that *first_fit* achieves the best balance between decision overhead and scheduling benefit.

Hard Rejection And Soft Scoring. Before computing any score, the policy applies four hard rejections: CPU capacity is insufficient, the predicted stall is shorter than the transfer time, no waiting request fits within the freed blocks, or GPU

Algorithm 1. Core Decision Logic of the Temporal Scheduler

```
procedure ShouldOffload(req)
    T_transfer ← EstimateTransferTime(req.blocks)
    T_fc ← PredictFCDuration(req.FC)
    if T_fc ≤ T_transfer then
        return false                          ▷ Stall too short.
    end if
    T_window ← T_fc − T_transfer
    N_capacity ← T_window × v_throughput
    waiting_req ← FindFirstFitRequest(N_capacity)
    return waiting_req is not null
end procedure
```

memory pressure is below a configurable threshold. If none of these conditions trigger, the policy computes a composite score that integrates GPU pressure, block fit quality, upload safety margin, and CPU capacity as positive signals, and penalizes offloading critical-path agents (using the Spatial Scheduler's priority metric), near-completion requests, and requests with high migration churn. The offload proceeds only if this score exceeds a threshold. The dominant positive term rewards stalls that are long relative to transfer time; the dominant penalty discourages offloading agents that the Spatial Scheduler has designated as critical. An emergency exception allows offloading even high-importance requests under severe GPU pressure when the stall margin is large.

This multi-factor gate encodes the empirical finding from §7.3: CPU offload is not universally beneficial—it helps only when a stalled cache can be converted into useful active work within a safe scheduling window.

4.3 Predictive Upload

Predictive upload completes the temporal scheduling cycle. If the upload starts only after a tool returns, the resumed request stalls on data transfer. If the upload reserves all destination GPU blocks too early, active requests lose memory before the cache is needed. TokenCake resolves this tension through gradual reservation guided by both urgency and application importance.

The scheduler ranks upload candidates by $P_{upload} = I + U$, where I is the normalized request importance from the Spatial Scheduler's priority metric; U is urgency based on proximity to the predicted function call completion time. Higher-priority and earlier-deadline requests are uploaded first. At each scheduling step, the scheduler computes an upload budget that protects critical waiting requests:

$$B_{upload} = \max(0,\ B_{gpu}^{free} - \max(0,\ D_{critical} - B_{shared}^{free})) \quad (3)$$

where $D_{critical}$ is the total block demand from critical waiting requests and B_{shared}^{free} is the number of free blocks in the shared pool. This ensures uploads never consume blocks that the Spatial Scheduler needs for critical-path agents.

Rather than allocating all destination GPU blocks at once—which risks evicting active requests under high memory utilization—the scheduler reserves at most half of each candidate's remaining deficit per scheduling step:

$$B_{reserve} = \min(B_{remain},\ \lceil \tfrac{B_{deficit}}{2} \rceil,\ B_{upload}) \quad (4)$$

This gradual approach amortizes allocation over several cycles, ensuring destination blocks are ready when the upload fires without displacing critical waiting work. The upload priority uses the same importance signal as the Spatial Scheduler's admission control (§5), so temporal and spatial decisions share a consistent notion of criticality.

5 The Spatial Scheduler

The spatial contention problem arises because agents of different criticality compete for the same GPU KV Cache memory. A simple FCFS memory allocation policy allows a non-critical agent that arrives first to occupy blocks that a critical-path agent needs, forcing the critical agent into costly recomputation and stalling the entire application. We call this phenomenon *critical inversion*. Prior agent-aware systems [14–16] recognize the importance of priority scheduling for LLM applications, but they operate at the request scheduling level and do not manage KV Cache memory allocation. Consequently, even with an optimal schedule, a non-critical agent can still exhaust GPU blocks and evict a critical agent's KV Cache.

The Spatial Scheduler solves critical inversion at the memory level rather than only at the scheduling level. It controls which agents can allocate GPU KV Cache blocks by dynamically partitioning memory into a shared pool that available to all agents and a reserved pool that accessible only to designated critical agents. This partitioning protects capacity for critical-path agents under memory pressure, ensuring that non-critical work cannot evict their KV Caches. The partition sizes adapt to runtime conditions through a feedback loop that combines static graph signals with dynamic runtime signals, as illustrated in Figure 8. The Spatial Scheduler periodically re-evaluates which agent types are critical, adjusts the total reserved capacity based on GPU memory pressure, and distributes reserved blocks among critical types proportionally to their importance and memory footprint.

Multi-GPU Support. TokenCake supports the same reservation policy in multi-GPU deployments. The Spatial Scheduler maintains per-GPU shared and reserved pools and uses the same agent priority metric to coordinate admission across devices. For tensor-parallel inference, a request is admitted only when the required KV Cache blocks can be reserved on all participating GPUs. The multi-GPU path keeps the policy unchanged and extends only the pressure snapshot with per-device free blocks, reserved blocks, and pending upload demand.

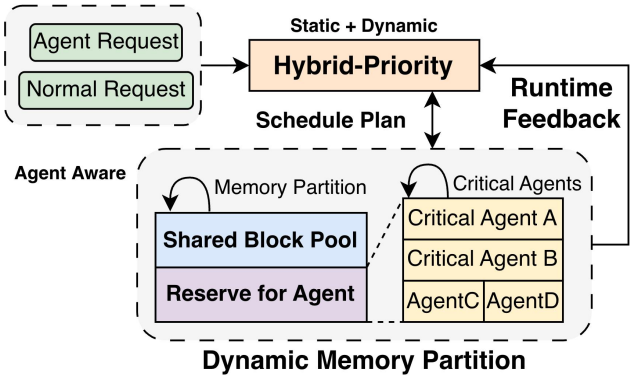


Figure 8. The Spatial Scheduler's dynamic memory partitioning feedback loop.

5.1 Dynamic Memory Partitioning

The Spatial Scheduler divides the GPU KV Cache memory into two regions: a globally shared pool available to all agents, and a reserved pool accessible only to the most critical agents. This two-pool structure protects critical work under contention while allowing non-critical work to proceed when memory is abundant. As shown in Algorithm 2, the partitioning adapts through a three-step update that runs periodically: adjusting the total reserved pool, selecting which agent types are critical, and distributing reserved capacity among them.

Step 1: Adjusting The Total Reserved Pool. The scheduler monitors GPU block usage and adjusts the fraction of total blocks allocated to the reserved pool. When usage is high, the reserved fraction increases to protect critical agents from contention. When usage is low, the reserved fraction decreases to return capacity to the shared pool. This feedback prevents over-reservation during low-load periods and under-reservation during memory pressure.

Step 2: Selecting Critical Agent Types. Before distributing reserved capacity, the scheduler determines which agent types are currently critical. It computes the agent-type score S_a (defined in §5.2) for every active agent type and designates the top fraction as critical. The current implementation uses a critical-agent ratio of 0.75 and caps the reserved pool at 30% of total blocks.

Step 3: Distributing Reserved Capacity Among Critical Agents. Once the total reserved pool size and the critical agent set are determined, the scheduler partitions reserved blocks among individual critical agent types. Each agent type's share is a weighted combination of two factors: its current GPU block usage (reflecting its actual memory footprint) and its priority score S_a relative to other critical agents. This ensures that agents which are both structurally important and memory-intensive receive proportionally larger reservations, while structurally important but memory-light agents still receive a non-zero protected allocation.

Algorithm 2. Dynamic Memory Reservation Update

```
procedure UpdateReservations
    N ← TotalGpuBlocks()
    usage ← CurrentGpuUsage() / N
                                ▷ Step 1: Adjust reserved pool size
    if usage ≥ high_watermark then
        ρ ← ρ+ step
    else if usage ≤ low_watermark then
        ρ ← ρ− step
    end if
    ρ ← Clamp(ρ, ρ_min, ρ_max)
                ▷ Step 2: Select critical agent types via S_a (Eq. 6)
    C ← TopFraction(active_types, S_a, critical_ratio)
                                ▷ Step 3: Distribute reserved blocks
    for all a ∈ C do
        share ← ½(GpuUsage(a)/N + S_a/∑_{a'∈C} S_{a'})
        reserved[a] ← share × ρ × N
    end for
end procedure
```

The current implementation uses a total reserved ratio that starts at 0.05, increases by 0.05 when GPU usage exceeds 0.75, decreases by 0.05 when usage falls below 0.40, and is clamped to $[0.05, 0.30]$. These thresholds make reservation conservative at low load and progressively stronger under memory pressure.

5.2 Hybrid Priority Metric

The Spatial Scheduler uses two levels of priority. A per-request priority P_{req} orders individual requests in the scheduling queue and determines which requests enter the next batch. A per-agent-type score S_a determines which agent classes receive reserved KV Cache capacity. Both combine static graph signals with dynamic runtime signals, but they operate at different granularities and use different formulas. Both metrics are enabled by the application-level context—the DAG structure, agent positions, and runtime history—that agent-agnostic systems lack.

Per-Request Priority. The scheduler refreshes P_{req} before every batch decision as a weighted sum over three dimensions:

$$P_{req} = \alpha_{\text{struct}} \cdot f_{\text{struct}} + \alpha_{\text{sync}} \cdot f_{\text{sync}} + \alpha_{\text{aging}} \cdot f_{\text{aging}}. \quad (5)$$

Structural importance (f_{struct}) captures how much downstream work a request unlocks by combining the node's depth and in/out-degree into a single score. *Synchronization pressure* (f_{sync}) boosts straggler branches in parallel workflows. At a join point, a lagging branch's priority increases inversely with its relative progress, preventing the merge node from becoming a bottleneck. *Temporal aging* (f_{aging}) prevents starvation and reduces tail latency by combining three signals: the fraction of the graph remaining, queue wait time, and a completion-pressure term that gives near-finished applications a final push.

Table 3. Pre-built FuncNode types in the frontend API.

Node	Description
FileReadNode	Read the contents of a specified file.
FileWriteNode	Write content to a specified file.
SearchNode	Perform a web search query.
FileQueryNode	Query files under a specified path.
DataAnalysisNode	Multi-stage analysis of large datasets.
UserConfirmNode	Request user confirmation.
ExternalTestNode	Use external test tools.

Agent-Type Score For Reservation. Unlike P_{req}, which scores individual requests, the per-agent-type score S_a aggregates across all active requests of a given type to decide whether the class as a whole deserves memory reservation. The score is a weighted sum over four dimensions:

$$S_a = w_1 P_a + w_2 U_a + w_3 H_a + w_4 G_a. \quad (6)$$

Structural priority (P_a) is a static priority, ensuring that a single high-criticality instance triggers protection for the entire type. *Runtime urgency* (U_a) captures how much the system has failed to serve type a, measured by preemption and waiting counts. Preemption receives a larger coefficient because it directly signals KV Cache capacity loss—the problem the Spatial Scheduler targets. *Recomputation cost* (H_a) protects types whose KV Caches are expensive to rebuild by log-compressing average token count, execution time, and throughput. *Graph context* (G_a) captures the average structural position of type a's active requests—depth and fan-in/out—ensuring that reservation reflects the agent type's role in the workflow DAG, not just its urgency or cost.

6 Implementation

TokenCake is a KV-Cache-centric serving system for multi-agent applications, composed of a front-end and an execution engine. It is implemented in approximately 9k lines of Python code and reuses some components from vLLM.

6.1 Frontend API

The frontend extends vLLM's OpenAI Chat Completion endpoint with a stateful graph registration interface. Users register a DAG of agents and function call nodes before submitting inference requests.

Table 3 lists the pre-built FuncNode types that the API provides. Each type bundles a default execution-time estimate and internal stage decomposition.

6.2 Function-Call Start And Finish Endpoints

The execution engine exposes two HTTP endpoints that drive the Temporal Scheduler. When an application begins a function call, it sends a request to `call_start` carrying the request identifier and an initial time estimate. This event transitions the request into a stalled state and makes it eligible for offload evaluation. When the function call completes, the application sends a request to `call_finish` carrying the request identifier and the actual elapsed time. This event marks the request as ready for upload and feeds the observed duration back to the per-function-type forecasting model in Equation 1.

Both endpoints are processed asynchronously by a unified `MCPManager` that maintains per-request lifecycle state. The manager maps each request to one of five states: running, pending-offload, offloaded, pending-upload, and uploaded. The two endpoints and the scheduling loop described next drive all state transitions.

6.3 CPU Migration Infrastructure

All KV Cache migration is issued asynchronously on a dedicated stream. Source GPU blocks are marked as pending-free immediately after the copy is issued and return to the free pool only after the transfer completes, preventing reallocation of blocks still being read.

vLLM V1 removed host-memory swap support and relies on recomputation for evicted KV Caches. TokenCake re-introduces a CPU block pool that maintains a lightweight free list recycling fixed-size blocks without returning them to the operating system. This avoids the costly system-allocator cycles that high-frequency offloading would otherwise induce, reducing worst-case CPU allocation latency from nearly a second to consistent sub-millisecond levels (§7.6).

The block pool owns the mapping between GPU blocks, CPU blocks, and block hashes. On offload, it links the GPU and CPU blocks, marks the KV Cache as CPU-resident, and inserts the block hash into a CPU prefix-cache index. On upload, it restores the GPU mapping and reuses the original GPU block when possible. The CPU prefix-cache index extends the standard lookup path: a CPU hit avoids recomputation but creates an H2D transfer entry that must complete before the request can run.

7 Evaluation

This section answers five questions about TokenCake. First, does TokenCake reduce end-to-end latency and improve GPU memory utilization compared to existing serving systems (§7.2)? Second, how much does each component contribute, and must agent scheduling and temporal offload be coordinated (§7.3)? Third, how does TokenCake compare with a remote-KV Cache baseline and an agent-aware execution baseline (§7.4)? Fourth, how sensitive is the Temporal Scheduler to tool-time variability and policy parameters (§7.5)? Fifth, is proactive offload practical and what overhead does it introduce (§7.6)?

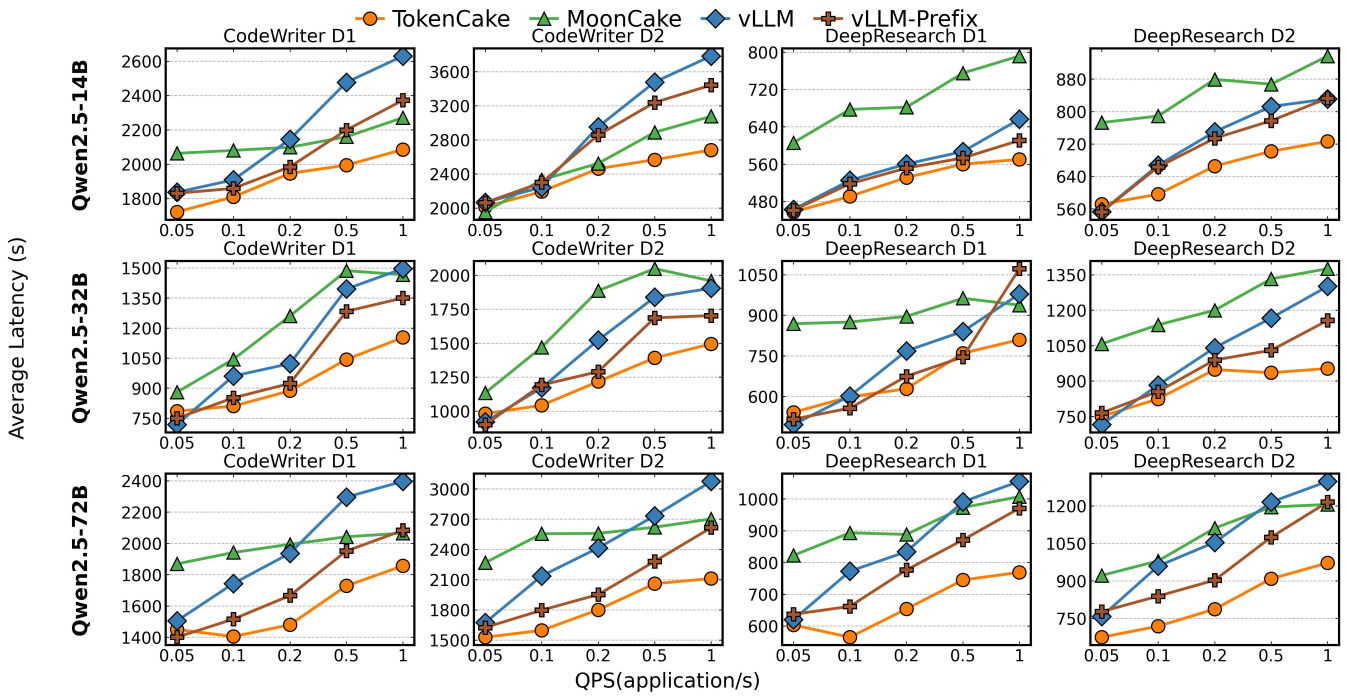


Figure 9. End-to-end application latency comparison of TokenCake, vLLM, and Mooncake. Each chart plots average latency against queries-per-second (QPS) for the specified application, model, and dataset.

7.1 Experimental Setup

Hardware and Models. We evaluate TokenCake on three configurations: Qwen2.5-14B on one NVIDIA A100 (80 GB HBM), Qwen2.5-32B on one NVIDIA H20 (96 GB HBM), and Qwen2.5-72B on two NVIDIA H20 GPUs with tensor parallelism degree 2. The 72B configuration exercises TokenCake's multi-GPU support, where the Spatial Scheduler coordinates KV Cache reservations across both GPUs. All configurations reserve 100 GB of CPU memory as the offload destination.

Benchmark Applications. We implement two representative multi-agent applications (Figure 1). Code-Writer (Figure 1a) orchestrates 11 agent types with frequent function calls to file I/O, search, and external test tools, creating high memory pressure from many concurrent KV Cache states. Deep Research (Figure 1b) models a research workflow with fewer agents but deeper dependency chains that stress critical-path optimization.

Workload Generation. User requests are synthesized from the ShareGPT [4] and AgentCode [2] datasets. Application arrivals follow a Poisson distribution at varying rates. External function calls are executed on a server, with tool endpoints deployed to match the MCP latency ranges in Table 1.

Baselines. We compare against four systems. **vLLM** (v0.8.6) is the standard KV-Cache-centric but agent-agnostic baseline. **vLLM-Prefix** adds prefix caching to vLLM for shared prompt reuse. **Mooncake** (v0.3.0-beta) is a remote-KV Cache design that tests whether general-purpose disaggregated caching suffices without function call awareness. **Parrot** is an agent-aware, compute-centric system that tests whether scheduling alone solves memory contention.

Metrics. We report end-to-end latency (average and tail percentiles) from application submission to final response, and GPU KV Cache utilization as the fraction of KV Cache blocks occupied over time.

7.2 End-to-End Performance

Latency Under Varying Load. Figure 9 shows average end-to-end latency as a function of request arrival rate across three model sizes (Qwen2.5-14B, 32B, 72B), two applications (Code-Writer, Deep Research), and two datasets (D1, D2). TokenCake consistently achieves the lowest latency across all configurations. At low load (0.05 QPS), TokenCake and vLLM perform comparably because memory contention is minimal. As the arrival rate increases, latency for vLLM and vLLM-Prefix grows steeply because both retain stalled agents' KV Caches in GPU memory, saturating capacity and forcing smaller batch sizes. Mooncake's remote caching provides partial relief at moderate load but cannot match TokenCake because it remains agent-agnostic. TokenCake's latency scales more gradually because the Temporal Scheduler offloads idle KV Caches during function call stalls, freeing blocks for active computation. At 1.0 QPS on Qwen2.5-14B Code-Writer D1, TokenCake reduces average latency by 47.06% compared to vLLM. The advantage holds across model sizes:

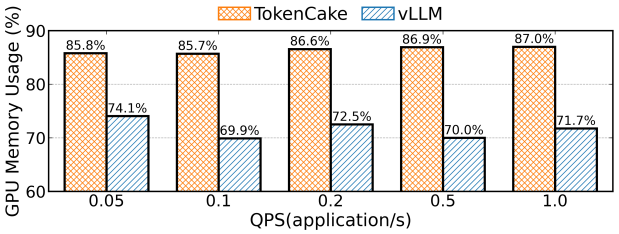


Figure 10. GPU KV Cache utilization on Qwen2.5-14B Code-Writer under varying load.

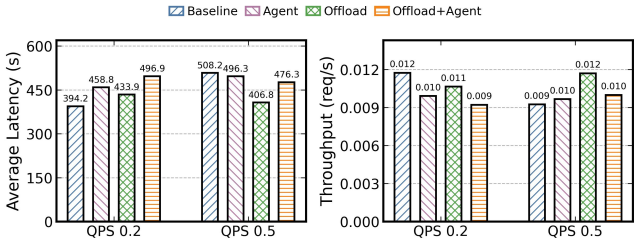


Figure 11. Component behavior at 0.2 and 0.5 QPS (Qwen2.5-14B Code-Writer, 20 apps, 0.5 GPU memory utilization).

on Qwen2.5-72B Code-Writer D2 at 1.0 QPS, TokenCake reduces latency by over 30%.

GPU Memory Utilization. The latency reduction is a direct consequence of better memory management. Figure 10 shows GPU KV Cache utilization on Qwen2.5-14B Code-Writer. TokenCake maintains utilization at 85.8–87.0% across all load levels, compared to 69.9–74.1% for vLLM, an improvement of up to 16.9 percentage points. The key difference is not total memory consumption but the utilization: TokenCake keeps occupied blocks allocated to active, computation-ready requests, whereas vLLM's blocks are partly held by idle KV Caches from stalled agents that block new requests from being scheduled. The same pattern appears on the larger Qwen2.5-32B model, where TokenCake raises utilization from 53.5% (vLLM) to 79.6% at 1.0 QPS.

7.3 Component Analysis

We isolate the contribution of each component using four modes: *baseline* (vLLM), *agent* (only the Spatial Scheduler), *offload* (only the Temporal Scheduler without agent awareness), and full TokenCake (both).

Scheduling and Offload Must Be Coordinated. On Code-Writer with Qwen2.5-14B (20 applications, 1.0 QPS, 0.5 GPU memory utilization), agent-aware scheduling alone reduces total latency from 502.2 s to 424.8 s (15.4%). Plain offload without agent awareness reduces total latency to 403.1 s but performs 11,339 offloads—more than double the swap volume of full TokenCake—and its average and P90 latencies (392.4 s, 397.1 s) remain close to baseline, indicating that indiscriminate offloading creates migration churn that delays some requests. TokenCake achieves the lowest latency across all metrics (344.6 s total, 313.7 s average, 328.3 s P90) while reducing swap volume by 51% compared to *offload* alone. Agent awareness allows the Temporal Scheduler to target offloads at non-critical stalled agents and protect critical-path caches, achieving better results with fewer migrations.

Load Dependence. Figure 11 shows component behavior at 0.2 and 0.5 QPS. At both load points, TokenCake achieves the lowest average latency and highest throughput. At 0.2 QPS, TokenCake reduces average latency from 496.9 s to 394.2 s (20.7%); at 0.5 QPS, it reduces latency to 406.8 s from 508.2 s (19.9%) and improves throughput by 33.3%. Across both load points, *agent* alone consistently outperforms *offload* alone, confirming that agent-aware scheduling provides a stronger standalone improvement than indiscriminate offloading. Combining both mechanisms yields a further reduction that neither achieves independently.

7.4 Remote-KV Cache and Agent-Aware Baselines

We compare TokenCake with Mooncake and Parrot, which represent the two sides of the design space that TokenCake bridges: KV-Cache-centric but agent-agnostic (Mooncake) and agent-aware but compute-centric (Parrot).

Mooncake. We connect Mooncake through the vLLM V1 Mooncake store connector with *kv_both* semantics. Figure 12 compares Baseline (vLLM), Mooncake, Offload (temporal scheduler only), and TokenCake at two arrival rates. At 0.2 QPS, Mooncake reduces average latency from 697 s to 524 s (24.8%) and increases throughput from 0.0069 to 0.0086 req/s, confirming that remote KV Cache reuse provides a meaningful benefit. TokenCake reduces latency further to 499 s, a 4.8% improvement over Mooncake. The modest gap at 0.2 QPS reflects low memory pressure: both systems have sufficient GPU blocks. At 0.5 QPS, where memory contention intensifies, the gap widens: Mooncake reaches 533 s while TokenCake achieves 384 s, a 28.0% reduction over Mooncake and 37.0% over Baseline. Throughput follows the same pattern: TokenCake reaches 0.0124 req/s, a 37.8% improvement over Mooncake's 0.0090 req/s. Offload alone (without agent awareness) performs worse than Mooncake at both load levels (577 s vs. 524 s at 0.2 QPS; 552 s vs. 533 s at 0.5 QPS), reinforcing the finding from §7.3 that indiscriminate offloading introduces migration churn. Only when offload is combined with agent-aware scheduling does TokenCake surpass Mooncake, because the Spatial Scheduler directs freed blocks to critical-path agents. The fundamental difference is that Mooncake remains agent-agnostic: it does not use function call events to predict idle intervals, nor does it prioritize caches based on graph criticality.

Parrot. Figure 13 compares TokenCake with Parrot on both applications across three arrival rates (0.1, 0.2, 1.0 QPS) using

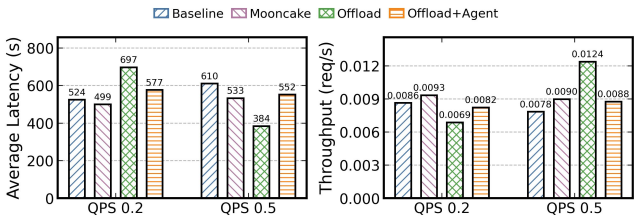


Figure 12. Mooncake comparison at 0.2 and 0.5 QPS.

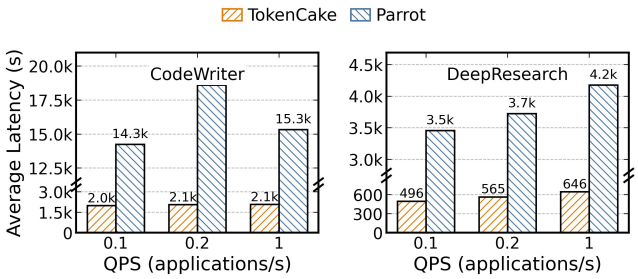


Figure 13. Parrot comparison across three QPS levels on Code-Writer (top) and Deep Research (bottom).

20 concurrent applications. On Code-Writer, Parrot's average latency ranges from 14.3k s to 18.6k s, while TokenCake achieves 2.0–2.1k s, a gap of 6.8–8.9×. On Deep Research, Parrot reports 3.5–4.2k s compared to TokenCake's 496–646 s, a gap of 6.5–7.1×. The gap persists across all load levels because the root cause is structural: Parrot does not manage KV Cache memory, so non-critical agents exhaust GPU blocks and cause eviction of critical agents' caches regardless of scheduling order. This comparison uses a different runtime (Parrot's own engine), so it serves as a system-scope check rather than a controlled experiment. It nonetheless confirms the paper's central claim: compute-centric scheduling cannot substitute for KV Cache-level memory management.

7.5 Sensitivity Analysis

We evaluate the Temporal Scheduler's sensitivity to two factors: tool-time prediction error and policy parameter choices.

Tool-Time Variability. We inject multiplicative noise into function call durations: at noise scale s, actual execution time is drawn from $[t \cdot (1-s),\ t \cdot (1+s)]$. Figure 14 shows the latency change of TokenCake relative to *agent*-only scheduling. With zero noise, TokenCake reduces latency by 14.8%. At noise 0.25, TokenCake regresses by 8.3% because mistimed uploads pass the gate's feasibility check but still cause harmful migrations. At noise 0.5, TokenCake recovers a 3.4% improvement because the larger errors cause scheduling windows to fail feasibility checks outright, so the gate correctly blocks most migrations. This non-monotonic pattern reveals that intermediate prediction error is the hardest regime: hard rejections protect against large errors, but marginal errors

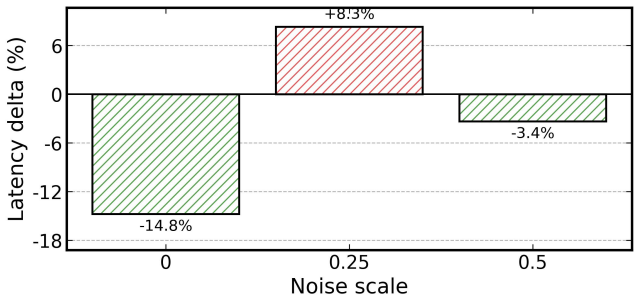


Figure 14. Latency delta of TokenCake vs. *agent*-only under tool-time noise. Negative means TokenCake is faster.

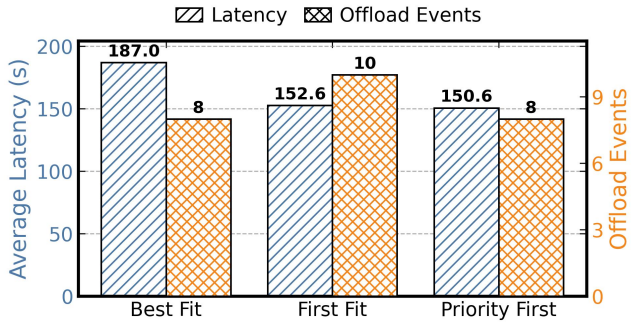


Figure 15. Average latency (left axis) and offload event count (right axis) for three request-selection policies.

that pass the gate impose disproportionate cost. Applications with highly variable tool latencies should provide wider confidence intervals so the gate can apply a larger safety margin.

Temporal Request Selection. We vary two policy choices: the request-selection strategy used by the opportunistic gate and the spatial pressure watermark. As shown in Figure 15, for request selection, *best_fit* (select the request whose demand best matches freed capacity) performs worst (187.0 s) because it reorders the queue and disrupts the Spatial Scheduler's priorities. *priority_first* (select the highest-priority request) achieves the lowest mean latency (150.6 s) but inflates the tail (P95: 173.2 s) by skipping small requests that would complete quickly. *first_fit* (select the first queue-order request that fits) achieves the best overall tradeoff: 152.6 s average, 164.7 s P95, and the highest throughput (0.058 req/s), because it preserves the queue order the Spatial Scheduler has already optimized. We use *first_fit* as the default.

Spatial Pressure Thresholds. For the spatial pressure watermark (Figure 16), aggressive thresholds (0.05, 0.06) trigger offloads frequently and produce similar latencies (~157 s). A higher watermark (0.08) rejects all offload candidates at this load level, achieving 107.5 s—a 32% reduction—because the gate correctly identifies that no waiting request can benefit from freed blocks. This does not mean zero offload is universally optimal; it confirms the selectivity principle that

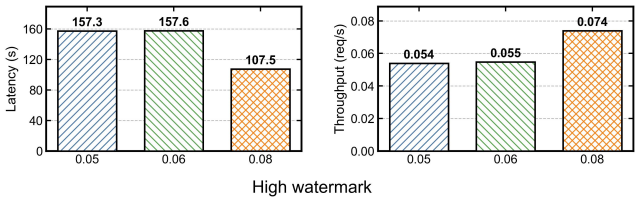


Figure 16. Sensitivity to spatial pressure thresholds.

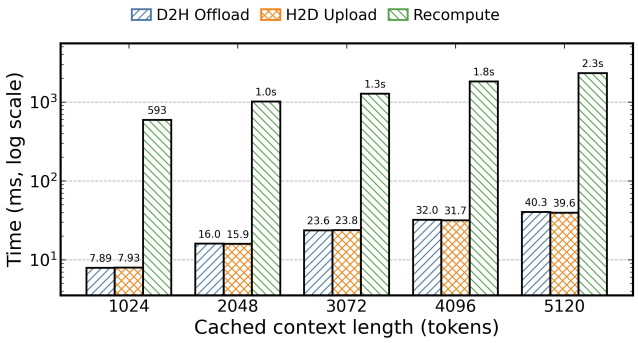


Figure 17. D2H offload, H2D upload, and recomputation time for Qwen2.5-14B on A100 PCIe.

offload should be conditional on whether freed blocks can admit useful active work.

7.6 Offload Overhead and Practicality

TokenCake assumes that preserving a stalled agent's KV Cache in CPU memory is cheaper than discarding and recomputing it. Figure 17 validates this assumption on the A100 PCIe platform used for Qwen2.5-14B. We measure cached context lengths from 1,024 to 5,120 tokens (64 to 320 blocks at 16 tokens/block, 3 MiB/block in bfloat16). At 4,096 tokens (256 blocks), offload takes 32.0 ms and upload takes 31.7 ms, for a 63.7 ms round trip. Recomputing the same context takes 1,815 ms, making recomputation 28.5× slower. Across all measured lengths, recomputation is 26.8–37.5× slower than a round-trip migration.

However, the round-trip cost still ranges from 15.8 to 79.8 ms, and under high concurrency unnecessary migrations can consume PCIe bandwidth and occupy GPU blocks that active requests need. This is why TokenCake uses the opportunistic gate rather than an always-offload policy.

8 Related Work

LLM Application-Aware Scheduling. Parrot [14], Autellix [16], and Hermes [15] model applications as dependency graphs to prioritize requests and mitigate head-of-line blocking. Teola [22] pipelines LLM and non-LLM stages to overlap tool execution with computation. These approaches are compute-centric: they optimize request orchestration but do not manage KV Cache memory, so a non-critical agent group can still exhaust GPU blocks and evict a critical-path agent's cache. TokenCake resolves this at the memory level through dynamic partitioning and proactive offload during function call stalls.

KV Cache Memory Management. PagedAttention [13] eliminates internal fragmentation via paged block allocation. Mooncake [21] disaggregates KV Cache storage from compute; CachedAttention [9] offloads session-level KV Caches to host memory; LMCache [27] extends effective cache capacity through reuse and storage techniques. These systems are agent-agnostic: they treat all KV Caches with equal importance and trigger offload reactively under memory pressure or session inactivity, without exploiting the predictable idle intervals that function calls expose or differentiating caches by graph criticality.

Positioning. As summarized in Table 2, agent-aware systems lack KV Cache memory management, and KV-Cache-centric systems lack application context. TokenCake occupies the missing point by co-optimizing temporal KV Cache lifecycle management and spatial memory reservation using the dependency graph and function call events exposed by multi-agent applications.

9 Discussion

Single-GPU Scope and Multi-GPU Extension. The current evaluation targets a single-GPU setting. The core principles of agent-aware KV Cache management—using graph structure and function call events to drive memory decisions—are not inherently tied to a single device. The Temporal Scheduler's offload policy could target a neighboring GPU over NVLink as a faster destination than CPU memory, creating a tiered memory hierarchy. The Spatial Scheduler's reservation logic could coordinate across devices to reduce cross-GPU critical inversion. A full multi-GPU performance evaluation remains future work.

Dynamic Graphs. The current evaluation uses static application graphs for reproducibility. In practice, some multi-agent applications contain dynamic edges where the LLM decides at runtime which downstream agent to invoke. TokenCake accommodates this at the request level: skipped branches never enter the scheduler, and new branches receive updated metadata from the frontend. The Spatial Scheduler's periodic re-evaluation of critical agent sets (§5.1) further adapts to changing workload composition. Integrating a probabilistic model of dynamic edges, as proposed in Hermes [15], could improve reservation accuracy for applications with highly variable graph structure.

10 Conclusion

Multi-agent LLM applications with external function calls expose two KV Cache problems that existing serving systems handle in isolation: *temporal underutilization*, where stalled agents' caches sit idle in GPU memory, and *spatial contention*,

where non-critical agents evict critical-path caches. Our key insight is that function call events make both the idle interval and the resume point of a KV Cache explicitly visible, and this temporal signal only yields sound memory decisions when combined with graph-level criticality.

TokenCake realizes this insight through two cooperating schedulers, a Temporal Scheduler that proactively offloads idle KV Caches during function call stalls and predictively uploads them before agents resume, and a Spatial Scheduler that reserves GPU capacity for critical-path agents using a hybrid priority metric. On representative multi-agent benchmarks, TokenCake cuts end-to-end latency by 47.06% and improves GPU memory utilization by 16.9% over vLLM under high load. Ablation studies further show that these gains demand coordination—agent-aware scheduling alone cannot reclaim memory from idle caches, and context-free offloading degrades performance—confirming that multi-agent LLM serving must treat KV Cache management as a joint scheduling and memory-lifecycle problem.

References

[1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiying Zhang. 2024. Infercept: Efficient intercept support for augmented large language model inference. *arXiv preprint arXiv:2402.01869* (2024).

[2] AlignmentLab AI. 2024. AgentCode: A dataset for code generated by LLM agents. Hugging Face Datasets. https://huggingface.co/datasets/AlignmentLab-AI/agentcode Accessed: 2025-09-01.

[3] Anomaly Innovations. 2025. OpenCode: The Open Source AI Coding Agent. https://opencode.ai. Accessed: 2026-05-15.

[4] anon8231489123. 2023. ShareGPT Vicuna Unfiltered. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered Hugging Face dataset.

[5] Anthropic. 2025. Claude Code: AI Coding Agent for Terminal and IDE. https://claude.com/product/claude-code. Accessed: 2026-05-15.

[6] Anthropic. 2025. Introducing Claude Research. https://claude.com/blog/research. Accessed: 2026-05-15.

[7] Anthropic, Inc. 2025. *Model Context Protocol Specification.* https://spec.modelcontextprotocol.io/specification/2025-08-20/ Accessed: 2025-08-20.

[8] ByteDance. 2025. TRAE: Collaborate with Intelligence — The Real AI Engineer. https://www.trae.ai. Accessed: 2026-05-15.

[9] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *Proceedings of the 2024 USENIX Conference on Usenix Annual Technical Conference* (Santa Clara, CA, USA) *(USENIX ATC'24)*. USENIX Association, USA, Article 7, 16 pages.

[10] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, Liuyi Yao, Hongyi Peng, Zeyu Zhang, Lin Zhu, Chen Cheng, Hongzhu Shi, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. AgentScope: A Flexible yet Robust Multi-Agent Platform. arXiv:2402.14034 [cs.MA] https://arxiv.org/abs/2402.14034

[11] Google. 2025. Gemini CLI: Open-Source AI Agent for the Terminal. https://geminicli.com. Accessed: 2026-05-15.

[12] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv:2308.00352 [cs.AI] https://arxiv.org/abs/2308.00352

[13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[14] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA. https://www.usenix.org/conference/osdi24/presentation/lin-chaofan

[15] Yifei Liu, Zuo Gan, Zhenghao Gan, Weiye Wang, Chen Chen, Yizhou Shan, Xusheng Chen, Zhenhua Han, Yifei Zhu, Shixuan Sun, and Minyi Guo. 2025. Efficient Serving of LLM Applications with Probabilistic Demand Modeling. arXiv:2506.14851 [cs.DC] https://arxiv.org/abs/2506.14851

[16] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. 2025. Autellix: An Efficient Serving Engine for LLM Agents as General Programs. arXiv:2502.13965 [cs.LG] https:

//arxiv.org/abs/2502.13965

[17] Microsoft. 2023. Microsoft 365 Copilot. Web page. https://www.microsoft.com/en-us/microsoft-365/enterprise/microsoft-365-copilot

[18] OpenAI. 2025. Codex: AI Assistant for Work and Code. https://chatgpt.com/codex/. Accessed: 2026-05-15.

[19] OpenAI. 2025. Introducing Deep Research. https://openai.com/index/introducing-deep-research/. Accessed: 2026-05-15.

[20] Jinghua Piao, Yuwei Yan, Jun Zhang, Nian Li, Junbo Yan, Xiaochong Lan, Zhihong Lu, Zhiheng Zheng, Jing Yi Wang, Di Zhou, Chen Gao, Fengli Xu, Fang Zhang, Ke Rong, Jun Su, and Yong Li. 2025. AgentSociety: Large-Scale Simulation of LLM-Driven Generative Agents Advances Understanding of Human Behaviors and Society. arXiv:2502.08691 [cs.SI] https://arxiv.org/abs/2502.08691

[21] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. arXiv:2407.00079 [cs.DC] https://arxiv.org/abs/2407.00079

[22] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Teola: Towards End-to-End Optimization of LLM-based Applications. arXiv:2407.00326 [cs.DC] https://arxiv.org/abs/2407.00326

[23] Google Gemini Team. 2025. Gemini Fullstack LangGraph Quickstart. https://github.com/google-gemini/gemini-fullstack-langgraph-quickstart. Accessed: 2025-09-23.

[24] Tencent Cloud. 2025. CodeBuddy: AI Intelligent Programming Assistant. https://www.codebuddy.ai. Accessed: 2026-05-15.

[25] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI] https://arxiv.org/abs/2308.08155

[26] Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. 2025. TradingAgents: Multi-Agents LLM Financial Trading Framework. arXiv:2412.20138 [q-fin.TR] https://arxiv.org/abs/2412.20138

[27] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. 94–109. doi:10.1145/3689031.3696098

[28] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. arXiv:2401.07339 [cs.SE] https://arxiv.org/abs/2401.07339